

Benchmarking Edge Deployment of DRL Navigation Policies: Latency, Compression, and Real-Robot Validation

Yunjia Chen*

School of Mathematical Sciences
University of Nottingham Ningbo China
Ningbo 315100, China

*Corresponding author: smyyc18@nottingham.edu.cn

Abstract—Autonomous mobile robot navigation maps sensor observations to motion commands within an on-board control loop. A deployable policy must be timely, compact, and competent, yet these properties are often evaluated separately. This paper defines a benchmark that combines model-only latency and artifact size, paired closed-loop task outcomes, and execution in a physical sensing and control stack. Each source of evidence has a stated limit on the claims it supports. We apply the benchmark to three DRL policies and one optimization-based planner using CPU and GPU execution, 999 paired simulation scenarios, 200 paired compression trials, and 50 formal trials on a WHEELTEC S100. Model-only latency for CNNTD3, SAC, and PPO ranges from 0.03 to 0.70 ms, but latency ordering does not track navigation success among the tested policies. ONNX Runtime provides a 6.3-fold speedup, whereas INT8 reduces size by 71% without a statistically significant task-level degradation. Fine-tuning recovers most of the performance lost at 50% pruning. On the robot, model-only inference is small relative to the 87–89 ms mean sensor-to-command path. FP32 and INT8 CNNTD3 each complete five center-blocking trials without observed contact; a low-clearance stress test defines the observed transfer boundary. The benchmark reports latency, size, and task outcome separately instead of combining them into one deployment score.

Keywords—deep reinforcement learning, edge deployment, inference latency, mobile robot navigation, model compression

I. INTRODUCTION

Autonomous mobile robot (AMR) navigation estimates motion commands that move a robot toward a goal while avoiding obstacles. This capability is used in service robots, logistics, and indoor transport, where sensing and control run on an on-board computer with limited memory and power. A deployed navigation system must produce safe, timely commands throughout an episode. Simulation success alone is insufficient because hardware, runtime software, sensing, and chassis dynamics affect physical execution.

AMR navigation may use optimization-based planners or learned policies. NeuPAN converts raw point clouds into a collision-constrained model-predictive optimization problem and has been validated on several robot platforms [1]. TD3 [2], SAC [3], and PPO [4] instead learn continuous-control policies through interaction. Learned policies offer a compact sensor-to-action mapping, but their deployment cost depends on

the model, execution backend, and target hardware. For sub-megabyte policies, accelerator dispatch and synchronization can be comparable to the network computation itself.

Runtime conversion and model compression are common ways to reduce this cost. McEnroe *et al.* evaluated compressed DRL policies for UAV obstacle avoidance on Jetson devices [5], and FERRO combines state reduction, transfer learning, and TensorRT deployment [6]. Post-training INT8 quantization reduces numerical precision, while structured pruning removes selected parameters or neurons. These methods can reduce storage or latency, but a smaller representation does not guarantee faster execution or unchanged navigation behavior. Their effects must be measured against the original policy on identical tasks.

Existing evaluations commonly report only one part of this deployment problem. Algorithm studies emphasize reward or success, runtime studies may report mean latency under device-specific conventions, and physical demonstrations may combine arrival, contact, and safety intervention into one label. The resulting measurements are difficult to compare across methods and platforms. Two operational questions remain: at the scale of a navigation policy, which deployment change reduces latency, and what task-level cost does it introduce? Answering these questions requires a protocol that fixes the timing convention, restores identical scenes for paired comparisons, and states what may be inferred from simulation and physical trials.

This paper makes two contributions. First, it defines a benchmark for edge navigation with fixed model-only timing, paired simulation, and physical-robot protocols, while retaining latency, size, and task outcomes as separate metrics. Second, it applies the benchmark to CNNTD3, SAC, PPO, and NeuPAN, then evaluates CNNTD3 after ONNX Runtime migration, INT8 quantization, structured pruning, and S100 deployment. These measurements show which changes affect latency or storage and delimit the task behavior preserved after deployment.

II. BENCHMARK DESIGN

A. Problem Definition and Evaluated Policies

Let π be a navigation policy, D its deployment form, and H the hardware and backend. The benchmark object is the

deployment configuration $c = (\pi, D, H)$. Warm, blocking, batch-one measurements characterize model-only latency and on-disk artifact size, but cannot establish navigation behavior. Restored, identical closed-loop scenes support paired task comparisons, but not claims about physical dynamics. Robot trials measure mean sensor-to-command latency and record arrival, contact, and safety activation; their claims remain limited to the tested layouts and trial counts. A geometrically matched scene provides the direct comparison between simulation and physical execution.

The evaluated learned policies are CNNTD3, a TD3 actor with a one-dimensional convolutional encoder, SAC, and PPO. Each implements $\pi : \mathbf{s}_t \mapsto (v_t, \omega_t)$ from a 185-dimensional state:

$$\mathbf{s}_t = [\tilde{\mathbf{l}}_t; d_t/10; \cos \theta_t; \sin \theta_t; 2v_{t-1}; (\omega_{t-1} + 1)/2] \in \mathbb{R}^{185} \quad (1)$$

where $\tilde{\mathbf{l}}_t \in [0, 1]^{180}$ is normalized LiDAR, d_t is goal distance, and θ_t is heading error. The evaluated checkpoints were produced in the same 10×10 m world with randomized goals, four RVO-driven moving obstacles, and three randomized static obstacles. They used a common training driver with algorithm-specific update schedules. Because optimizer updates were not equalized across algorithms, the task results characterize these checkpoint instances rather than the algorithms in general. Simulation actions were mapped as $v_t = (a_t^{(v)} + 1)/4$ and $\omega_t = a_t^{(\omega)}$. The robot additionally used linear and angular scales of 0.8 and 1.0. The deployed CNNTD3 files were hash-verified against the evaluated exports.

B. Execution-Efficiency Protocol

All platforms used the same script and input shape: 50 warm-up iterations followed by five independent batches of 1000 timed inferences. We report pooled mean, p50, p95, and p99. CUDA and MPS were synchronized after each forward pass; CPU required no device synchronization. Measurements therefore represent warm, batch-one, blocking model-only latency with device-resident inputs, excluding transfer and the ROS chain. Hardware, backend, and runtime effects are not separated. Off-robot DRL measurements used an Apple M5 with Python 3.11.15 and PyTorch 2.13.0, and an RTX 4070 12-GB host under Ubuntu 22.04 with PyTorch 2.13.0+cu130; compression tests used ONNX Runtime 1.28.0.

Batch size one reflects sequential control rather than throughput inference, and synchronization prevents timing from ending at asynchronous GPU submission. Cross-batch stability is the sample standard deviation of the five batch means. Robot sensor-to-command latency additionally includes scan arrival, state construction, and command publication, so it is reported separately from model-only latency.

C. Task-Effectiveness Protocol

All methods used common scenes under seed 42 (500 episodes) and seed 7 (499 episodes), totaling 999 paired scenarios; one invalid episode was commonly skipped. During scene generation, a fixed two-step feasibility probe filtered unsafe initial states, with acceptance rates of 65.2% and

69.5%. Evaluation restored the pre-probe pose and random-state snapshot, then terminated on arrival within 0.3 m, collision, or 500 steps. Exact paired McNemar tests were used for method comparisons.

NeuPAN used its official repository and pretrained DUNE weights. Kinematic adaptation, 11 single-parameter scans, and 40 rounds of published LON failure-backpropagation tuning reached 2.5% success. This result is restricted to the tested interface and environment because adaptation effects cannot be excluded.

D. Deployment-Optimization Protocol

Structured pruning removes MLP neurons by L_2 -norm ranking at 10%, 30%, and 50%, followed by TD3 fine-tuning. This implementation adapts the general pruning-and-retraining practice in [7], [8] to the policy MLP. Static INT8 quantization uses real navigation states for calibration and ONNX Runtime CPU inference. Its integer-inference rationale follows [9]. Original and compressed policies were evaluated in one process on 200 paired deterministic scenarios using exact McNemar tests.

E. Physical-Robot Protocol

Deployment used a WHEELTEC S100 with Jetson Orin Super, ROS1 Noetic, PyTorch 1.14.0a0+nv23.02, ONNX Runtime 1.16.x, a 12-Hz 360-degree LiDAR, and 3.3-Hz control. The front 180-degree scan supplied the state in (1). A safety layer stopped the robot below 0.20 m frontal clearance. Arrival was an odometry distance below 0.3 m; contact and shield activation were recorded separately. The path-overlap stress test additionally required no contact and physical endpoint error no greater than 0.40 m. Ten formal trials were run per scenario. A three-run FP32 go/no-go pilot was excluded from the center-blocking statistics. A separate geometrically matched C1 simulation was not included in the 999-scenario benchmark.

III. RESULTS

A. Execution Regime

Table I and Fig. 1 show DRL mean latencies between 0.03 and 0.70 ms on the M5 and RTX platforms. CPU distributions are concentrated and stable (cross-batch sample standard deviation at most 0.0016 ms). MPS has wider tails and higher batch variance, whereas RTX 4070 is comparatively concentrated. At this model scale, the timing is consistent with dispatch, synchronization, and kernel-launch overhead accounting for much of the measured latency. On the tested stacks, CPU execution is faster and more predictable. NeuPAN averages 10.255 ms and has a highly right-skewed distribution, with median decisions below 0.05 ms and upper-tail solves near 30 ms.

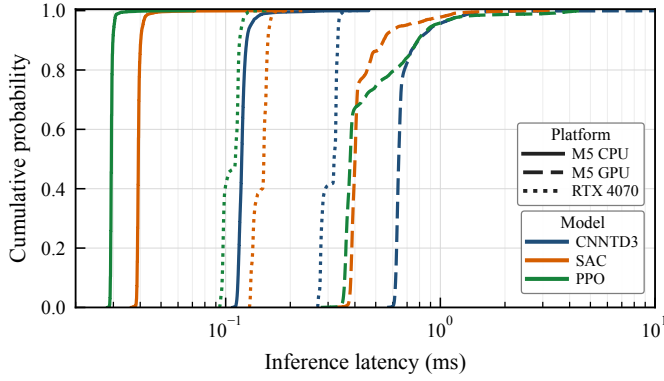


Fig. 1. Empirical latency distributions for three DRL policies on the evaluated CPU and GPU platforms.

TABLE I
MODEL-ONLY LATENCY (MS): MEAN/P50 ABOVE P95/P99; FIVE POOLED 1000-INFEREANCE BATCHES PER DRL CELL.

Platform	CNNTD3	SAC	PPO	NeuPAN
M5 CPU	.120/.119 .131/.151	.040/.039 .041/.045	.030/.029 .030/.032	—
M5 GPU	.696/.640 .944/1.355	.448/.401 .684/1.149	.513/.380 .929/3.038	—
RTX 4070	.307/.320 .335/.346	.146/.151 .160/.166	.107/.111 .119/.125	—
Host CPU	—	—	—	10.255/.048 29.084/30.141

B. Latency and Task Competence

Table II gives the paired 999-scenario results. SAC performs best among the evaluated checkpoints, followed by CNNTD3 and PPO. As specified in Section II-A, this ordering is conditional on the training schedules and reward setting. NeuPAN failures are timeout-dominated, consistent with a mismatch between the environment and method configuration rather than a general algorithm ranking. Adjacent differences are supported by exact McNemar tests: SAC–CNNTD3 $p \approx 2.44 \times 10^{-27}$, CNNTD3–PPO $p \approx 1.57 \times 10^{-82}$, and PPO–NeuPAN $p \approx 1.6 \times 10^{-76}$.

The Wilson 95% confidence intervals are 81.3–85.8% for SAC, 65.8–71.6% for CNNTD3, 28.1–33.9% for PPO, and 1.7–3.7% for NeuPAN. These intervals describe marginal precision, whereas McNemar uses paired outcomes. NeuPAN’s low collision rate occurs with 92.0% timeouts and therefore does not indicate overall reliability.

TABLE II
NAVIGATION OUTCOMES ON 999 PAIRED SCENARIOS.

Method	Success	Collision	Timeout	Steps
CNNTD3	68.8%	8.5%	22.7%	146
SAC	83.7%	7.1%	9.2%	103
PPO	30.9%	37.0%	32.0%	268
NeuPAN	2.5%	5.5%	92.0%	461

C. Deployment-Optimization Trade-offs

Moving from PyTorch to ONNX Runtime yields a 6.3-fold speedup (0.1204 to 0.0191 ms) in Table III; this is a joint deployment-stack effect. INT8 reduces size by 71% and changes success from 72.5% to 72.0% (discordant pairs 4/3, $p = 1.0$), so strict losslessness is not claimed. Fifty-percent pruning significantly reduces success to 61.0% ($p \approx 0.00029$). Fine-tuning significantly recovers it to 70.0% ($p \approx 0.011$), with no significant difference from baseline ($p \approx 0.44$). Pruning primarily reduces size, while its small latency differences are consistent with fixed overhead at this model scale.

TABLE III
CNNTD3 COMPRESSION TRADE-OFFS ON 200 PAIRED SCENARIOS.

Version	Size (MiB)	Latency (ms)	Success
Baseline	.538	.1204	72.5%
Pruned 10%	.444	.1178	72.5%
Pruned 30%	.286	.1173	76.0%
Pruned 50%	.164	.1221	61.0%
Pruned 50% + FT	.164	.1177	70.0%
ORT FP32	.524	.0191	72.0%
ORT INT8	.157	.0282	72.0%

D. Physical-Robot Consistency Boundary

Table IV reports pooled on-board model-only latency of 1.539, 0.092, and 0.100 ms for TorchScript FP32, ORT FP32, and ORT INT8, respectively. The TorchScript result includes all five preregistered batches; its final three batches averaged approximately 0.53 ms, but were not selected post hoc. In dedicated ROS runs, mean sensor-to-command latency was 89.1, 87.2, and 87.2 ms for the same three formats, consistent with approximately one 12-Hz scan period plus callback processing and within the 3.3-Hz control cadence.

TABLE IV
ON-BOARD CNNTD3 MODEL-ONLY LATENCY IN MS (FIVE POOLED BATCHES OF 1000 ITERATIONS).

Format	Mean	p50	p95	p99
TS FP32	1.539	.512	3.877	4.842
ORT FP32	.092	.088	.121	.171
ORT INT8	.100	.098	.105	.121

Table V and Fig. 2 summarize the exploratory trials. Straight and center-blocking navigation both achieve 10/10 odometry-defined arrival. The center-blocking trials are contact-free for FP32 (5/5) and INT8 (5/5), with no shield activation. Combined with 20/20 success in the separate, geometrically matched C1 simulation configuration, these results provide evidence of basic autonomous obstacle avoidance on the tested robot, with no observed task-level degradation from INT8 at this exploratory scale. The remaining trials define the scope of this result. Both side-obstacle scenes achieve 10/10 arrival, although four rear contacts occur on the left versus none on the right (two-sided Fisher $p \approx 0.087$). The path-overlap stress test yields only 2/10 odometry arrivals and 0/10 composite successes.

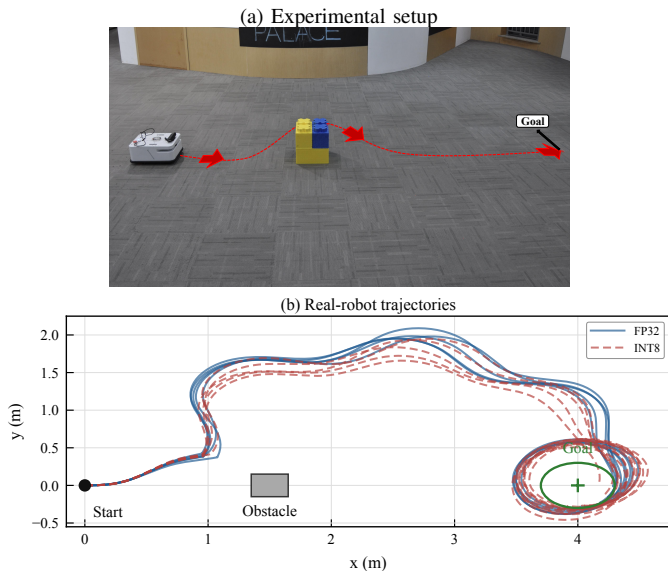


Fig. 2. Real-robot center-blocking validation: (a) experimental scene with a schematic task path and (b) odometry trajectories for five FP32 and five INT8 formal trials. All trials bypassed the obstacle, had no contact according to on-site observation logs, and reached the odometry-defined goal region.

TABLE V
EXPLORATORY REAL-ROBOT RESULTS; COLUMNS ARE NOT MUTUALLY EXCLUSIVE.

Scenario	N	Arrival	Contact	Shield
Straight	10	10/10	0/10	0/10
Center-blocking	10	10/10	0/10	0/10
Side obstacle, left	10	10/10	4/10	0/10
Side obstacle, right	10	10/10	0/10	0/10
Path-overlap stress	10	2/10	2/10	8/10

IV. DISCUSSION

The measurements are consistent with an overhead-bound regime for these sub-megabyte policies. CPU execution is faster and more predictable on the tested stacks, although larger or fused networks may change this balance. ONNX Runtime provides the main latency gain, whereas INT8 primarily reduces storage. Pruning reduces size but requires fine-tuning to recover most of the loss at 50%, unlike the larger-model gains in [5].

The physical trials support center-blocking avoidance on the tested S100, not general navigation reliability. Arrival, contact, and shield activation remain separate because arrival can coexist with contact or inefficient motion. Five of ten center-blocking trials show post-hoc goal-approach looping, but zero commands after arrival identify an efficiency limitation rather than a stopping-logic failure. The simulator omits chassis response, sensor error, localization drift [10], and the observed rear-body sweep. Bag data indicate angular reversal and stopping lags of approximately 0.14–0.49 s.

C1 verifies the center-blocking geometry under idealized execution; the robot adds sensing, localization, chassis response, and the safety rule. Agreement supports transfer for that layout, but not for low clearance. The frontal shield cannot cover rear

sweep and may prevent progress after repeated activation, so its activation is reported separately from avoidance.

The NeuPAN result remains configuration-specific. Although official code, weights, and LON tuning were used, the environment, LiDAR interface, and speed limit differ from its intended deployments. The result is not a universal ranking.

V. CONCLUSION

The benchmark fixes the timing convention, pairs task comparisons on identical scenes, and restricts physical claims to the layouts tested. Its application shows that sub-millisecond inference does not imply stronger navigation performance, ONNX Runtime rather than numerical precision supplies the main latency gain, and equal-size compressed artifacts need not preserve the same behavior. INT8 reduces size by 71% without a statistically significant task-level drop in 200 paired scenarios, while fine-tuning recovers most of the loss at 50% pruning. On the S100, FP32 and INT8 each complete five center-blocking trials without contact according to on-site logs; low-clearance trials mark the boundary of this result. By separating model-only efficiency, paired closed-loop outcomes, compression effects, and physical-robot evidence, the protocol also provides a reporting structure for edge-navigation deployment studies. The protocol, rather than a single platform ranking, is the transferable contribution.

REFERENCES

- [1] R. Han, S. Wang, S. Wang, Z. Zhang, J. Chen, S. Lin, C. Li, C. Xu, Y. C. Eldar, Q. Hao, and J. Pan, “NeuPAN: Direct point robot navigation with end-to-end model-based learning,” *IEEE Transactions on Robotics*, vol. 41, pp. 2804–2824, 2025.
- [2] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1587–1596.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1861–1870.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] P. McEnroe, S. Wang, and M. Liyanage, “Towards latency efficient DRL inference: Improving UAV obstacle avoidance at the edge through model compression,” in *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, 2024, pp. 4242–4249.
- [6] P. McEnroe, S. Wang, and M. Liyanage, “FERO: Efficient deep reinforcement learning based UAV obstacle avoidance at the edge,” *IEEE Open Journal of the Computer Society*, vol. 6, pp. 1378–1389, 2025.
- [7] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *International Conference on Learning Representations*, 2016.
- [8] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient ConvNets,” in *International Conference on Learning Representations*, 2017.
- [9] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [10] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.